VULNERABILITY ASSESSMENT REPORT

Task 1: Web Application Security Testing

Target Website: http://testphp.vulnweb.com

Prepared By: Akshay Jain
Cybersecurity Intern – Future Interns

Assessment Period:
05 March 2026 – 09 March 2026

Tools Used:

Tool	**Purpose**
Nmap	Network port scanning
OWASP ZAP	Web vulnerability scanning
Browser DevTools	HTTP header analysis
SecurityHeaders.com	Security header evaluation

Executive Summary

This report presents the results of a vulnerability assessment conducted on the website **testphp.vulnweb.com**. The objective of this assessment was to identify potential security weaknesses that could expose the web application to cyber threats.

The assessment was performed using **passive scanning techniques** in accordance with ethical testing guidelines. No exploitation or intrusive testing was conducted.

Several security misconfigurations were identified, including:

- Missing security headers
- Information disclosure through HTTP headers
- Insecure cookie configuration
- Unencrypted HTTP communication
- Exposed open ports

These vulnerabilities could potentially allow attackers to gather sensitive information or exploit weaknesses in the application.

Appropriate remediation measures have been recommended to improve the overall security posture of the application.

Scope of Assessment

The vulnerability assessment was conducted on publicly accessible pages of the following target:

http://testphp.vulnweb.com

Ethical Guidelines Followed

The assessment strictly followed ethical cybersecurity practices:

Allowed activities:

- Passive scanning
- HTTP header analysis
- Configuration review
- Security header inspection

Prohibited activities (not performed):

- Exploitation of vulnerabilities
- Login bypass attempts
- Brute force attacks
- Denial-of-Service attacks

The assessment was conducted purely for **educational and security auditing purposes**.

Methodology

The vulnerability assessment followed a structured methodology consisting of the following phases:

1. **Reconnaissance**
    - Identification of web technologies and server information.
2. **Configuration Analysis**
    - Examination of HTTP response headers.
    - Identification of exposed server information.
3. **Security Header Analysis**
    - Evaluation of missing or misconfigured security headers.
4. **Passive Vulnerability Scanning**
    - OWASP ZAP was used to identify security misconfigurations.
5. **Network Analysis**
    - Nmap was used to identify exposed network services.

The findings were then categorized according to their **risk level**.

Target Information

Website: http://testphp.vulnweb.com

Observed Technologies:

Technology	Purpose
PHP	Backend scripting language
HTTP Server	Web server hosting application

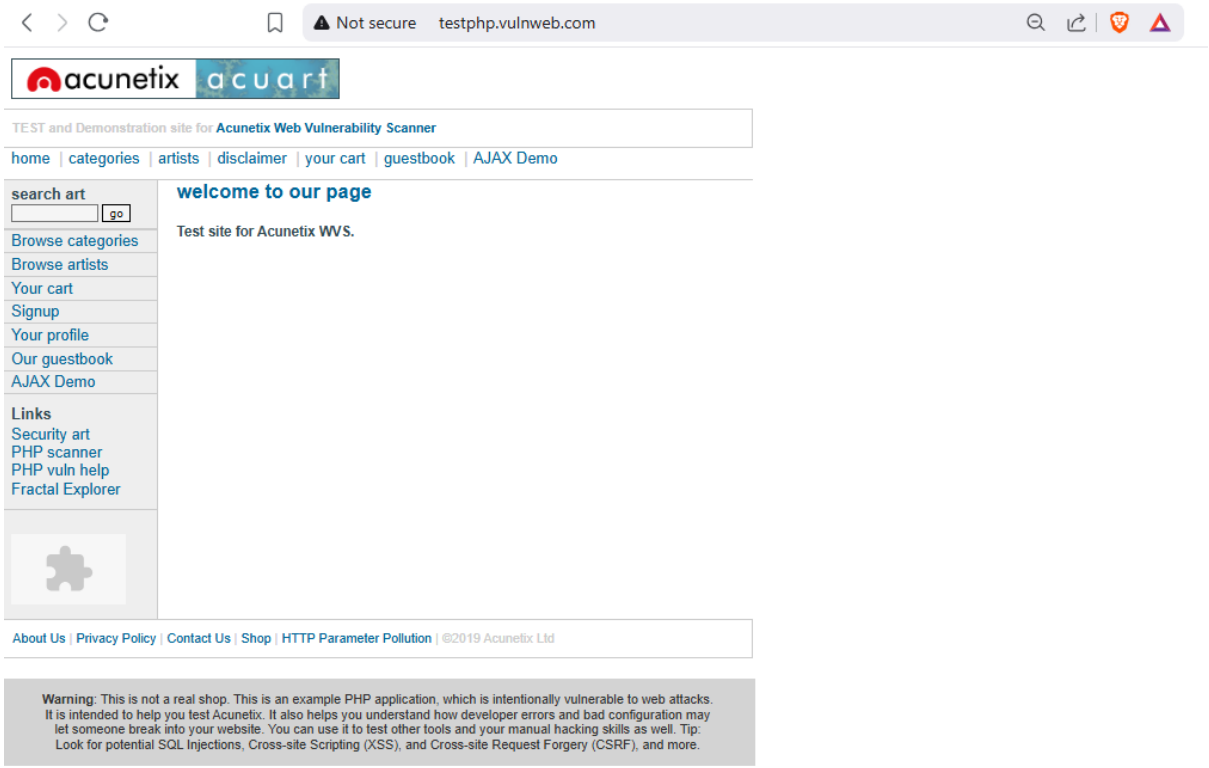

Figure 1 — Target Web Application

Vulnerability Table

ID	Vulnerability	Risk	CVSS Severity
V1	Unencrypted HTTP Communication	Medium	5.3
V2	Insecure Cookie Configuration	Medium	5.0
V3	Missing Security Headers	Low	3.1
V4	Information Disclosure	Low	2.6
V5	Open Ports Exposure	Low/Informational	2.0

Vulnerability 1 (HTTP Instead of HTTPS)

V1 — Unencrypted HTTP Communication

Description:

The web application uses HTTP instead of HTTPS for communication.

http://testphp.vulnweb.com

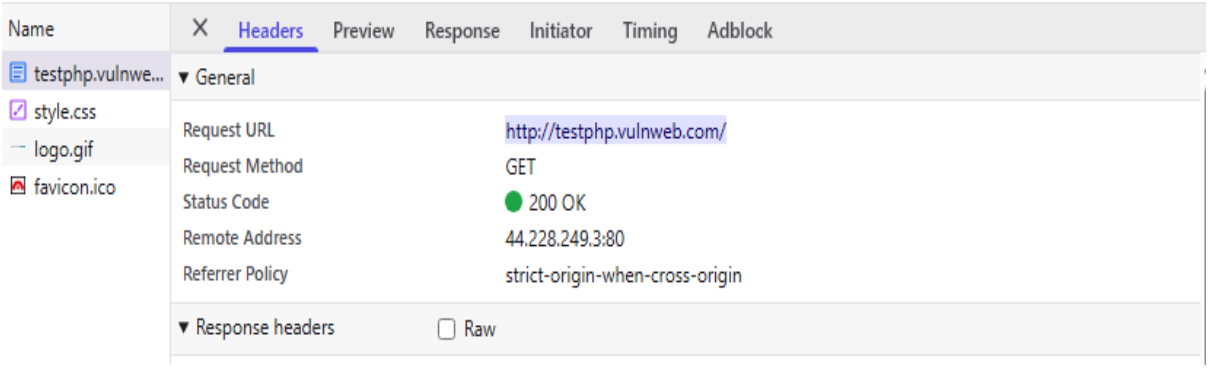

Figure 2 — Website Using HTTP

Impact:

Unencrypted HTTP communication may allow attackers to intercept sensitive data transmitted between users and the server.

Risk Level:

Medium

Remediation:

Implement HTTPS using SSL/TLS certificates and redirect HTTP traffic to HTTPS.

Vulnerability 2 (Insecure Cookies)

V2 — Insecure Cookie Configuration

Description:

Cookies used by the application lack important security flags.

Important cookie security attributes include:

- Secure
- HttpOnly
- SameSite

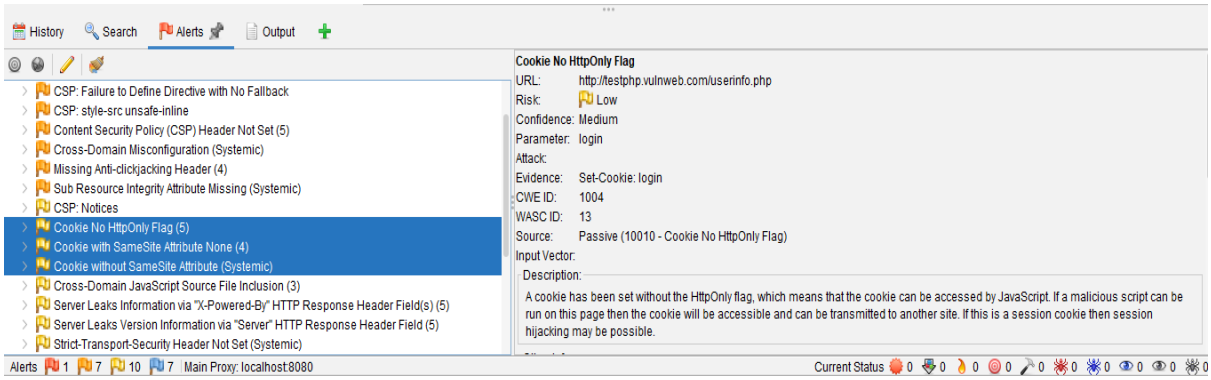

Figure 3 — Cookie Security Issue Identified

Impact:

Improper cookie configuration could allow attackers to steal session identifiers and hijack user sessions.

Risk Level:

Medium

Remediation:

Configure cookies with security flags.

Example:

Set-Cookie: sessionid=xyz; Secure; HttpOnly; SameSite=Strict

Vulnerability 3 (Missing Security Headers)

V3 — Missing Security Headers

Description:

Security headers help protect web applications against common attacks such as cross-site scripting and clickjacking. The assessment identified several missing security headers.

Missing headers include:

- X-Frame-Options
- Content-Security-Policy
- X-Content-Type-Options

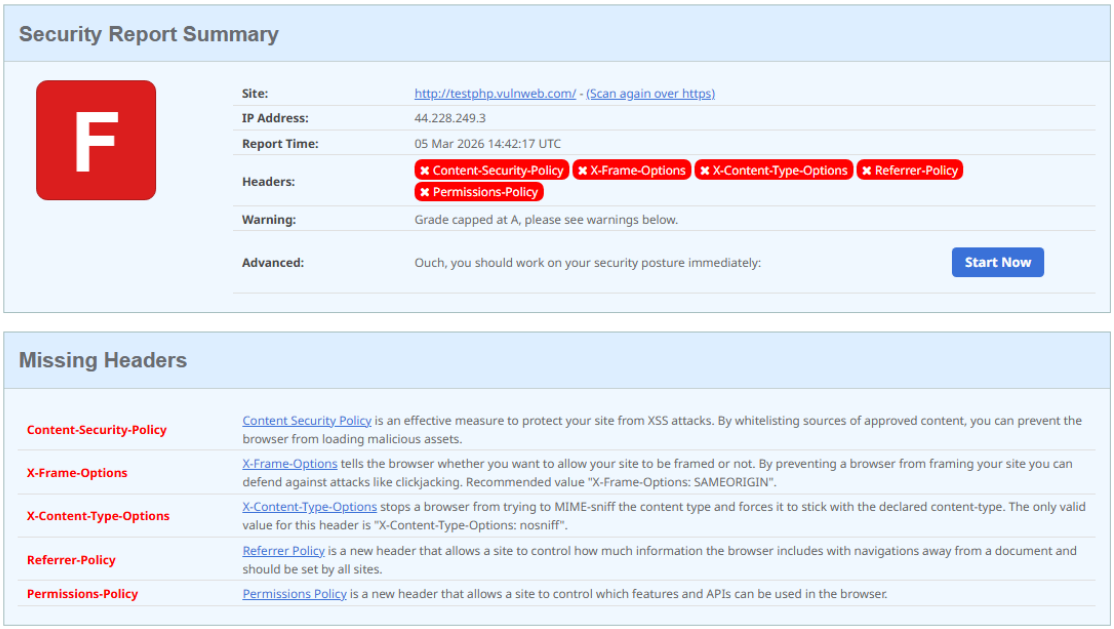

Figure 4 — Security Headers Scan Result

Impact:

Without these headers, attackers may be able to perform client-side attacks such as clickjacking or script injection.

Risk Level:

Low

Remediation:

Add the following headers to the server configuration:

X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'

Vulnerability 4 (Information Disclosure)

V4 — Information Disclosure via HTTP Headers

Description:

Information disclosure occurs when a web server reveals sensitive technical details in HTTP response headers. During the assessment, the web server exposed information about the underlying technology stack.

Evidence:

The following headers were observed:

Server: nginx/1.19.0
X-Powered-By: PHP

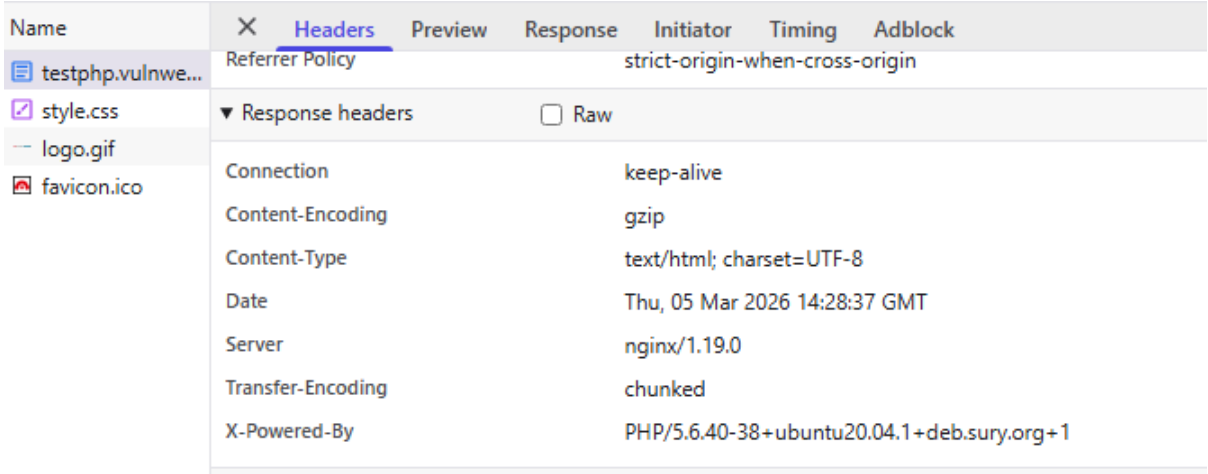

Figure 5 — HTTP Response Headers Revealing Server Information

Impact:

Disclosure of server technology may help attackers identify potential vulnerabilities associated with specific software versions.

Risk Level:

Low

Remediation:

Server configuration should be modified to hide version details.

Example:

ServerTokens Prod
ServerSignature Off

Vulnerability 5 (Open Ports)

V5 — Open Ports Detected

Description:

Nmap scanning identified open ports on the server.

Command used:

nmap -sT testphp.vulnweb.com

```
PORT   STATE SERVICE
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 455.61 seconds
```

Figure 6 — Nmap Scanning

Impact:

Open ports expose services to the internet, which may increase the attack surface if additional vulnerabilities exist.

Risk Level:

Low/Informational

Remediation:

Close unnecessary ports using firewall rules. And use 443 instead of 80 (https instead of http)

Recommendations

To improve the security posture of the application, the following actions are recommended:

1. Implement HTTPS across the entire website.
2. Configure proper security headers.
3. Hide server information in HTTP responses.
4. Secure cookies using HttpOnly and Secure attributes.
5. Restrict exposed services using firewall rules.

Conclusion

The vulnerability assessment identified several security misconfigurations within the target web application. While no critical vulnerabilities were identified during passive testing, the discovered weaknesses could still increase the attack surface of the application.

By implementing the recommended remediation measures, the organization can significantly improve the security posture of the web application and reduce potential cyber risks.